

EVOPROMPT-F: USING LLMs IN EVOLUTIONARY ALGORITHMS WITH SELF-EXPLANATION MECHANISMS

Nguyen Bang Anh^{1*}, Dang Nguyen Duy Truc¹, La Nguyen Thi
Tram¹, Luong Thi Hao¹

¹University of Engineering and Technology, Vietnam National
University.

Abstract

Recently, Large Language Models (LLMs) are applied in many Natural Language Processing applications and show high performance. However, the quality of generated output depend strongly on the input prompt. To design an effective prompt, it usually require domain knowledge and trial-and-error process, which is a big challenge for non-expert users. Therefore, automatic prompt optimization becomes an important research problem to improve the usage of LLMs without human effort.

In this paper, we propose a framework for discrete prompt optimization. We combine dynamic few-shot learning with Evolutionary Algorithms (EAs) to solve the mentioned problems. The main idea is using LLMs to perform evolutionary operators, such as mutation and crossover, on both instruction text and in-context examples. We start the process from a set of heuristic prompts. By this method, the final prompt can combine good few-shot demonstrations with structured explanations.

In order to evaluate the effectiveness of the proposed approach, we conduct experiment on Big-Bench Hard (BBH) dataset with Gemma model. The experiment results show the advantages of our method. Specifically, when applying Genetic Algorithms (GA), the performance on the *Logical Deduction (Seven Objects)* task increase from 55.5% (baseline) to 73.5%. Beside that, using Differential Evolution (DE) to optimize weights in CNN-based GTSRB algorithm also help increase accuracy to 77.5%. These results prove the effectiveness of combining evolutionary optimization with open-weight language models like Gemma.

Source code: <https://github.com/bangils37/EvoPrompt.git>.

Keywords: Evolutionary Algorithms, LLMs, Automatic Prompt Optimization, Self-Explanation Mechanism

1 Introduction

Large Language Models (LLMs) have shown outstanding performance in several natural language processing applications. Nonetheless, the quality of their outputs is greatly influenced by the input prompt. Because creating an effective prompt often

requires a considerable deal of skill and repeated trial and error, it may be difficult for non-expert users. Therefore, increasing the effectiveness and utility of LLMs without human involvement has made autonomous prompt optimization an important area of study.

Several approaches have been proposed to address this problem [1] [2], including evolutionary-based methods like EvoPrompt, local search, and random prompt editing. These methods have two significant disadvantages despite the promising results. First, the prompt search space is often limited by small initial populations and simple, essentially random modifications. Second, the optimization approach does not fully exploit the reasoning and explanation capabilities of LLMs, resulting in prompts that are not properly contextualized to support difficult reasoning tasks.

To address these issues, this paper proposes a method called EvoPrompt-F. The proposed approach combines evolutionary algorithms with prompt context enrichment inspired by few-shot prompting. Specifically, EvoPrompt-F uses the LLM to generate explanations for crossover and mutation processes as well as samples from the development set during the evolution process, and then combines these explanations into the prompt. This strategy expands the search field and improves the quality of the provided suggestions.

We evaluate the proposed method on the logical deduction seven objects task from the Big-Bench Hard (BBH) benchmark using the Gemma model. The experimental results show that EvoPrompt-F achieves significantly higher accuracy than the baseline method, demonstrating the effectiveness of combining evolutionary optimization with context enrichment.

2 Background

2.1 The Discrete Prompt Optimization Problem

In the context of in-context learning, the prompt plays a very important role as the main mechanism to control the reasoning ability and the quality of output of an LLM. Therefore, the performance of the model and the precision of the responses are impacted directly by how we choose the prompt. In this section, we define the following notations to describe the problem:

- X is defined as the space of input data.
- Y is defined as the space of output data.
- $\mathcal{P}$ represents the space of discrete prompts. In this space, each prompt $p \in \mathcal{P}$ is a sequence of tokens in natural language.

Ideally, we can represent a Large Language Model (LLM) as a mapping function as below:

$$f : \mathcal{P} \times X \rightarrow Y.$$

The model will produce a prediction $y \in Y$ when we give it a prompt $p \in \mathcal{P}$ and a data sample $x \in X$. In common practice, the input data and the prompt are usually concatenated directly. We denote this as $p \oplus x$, where the symbol $\oplus$ stands for the string concatenation operator.

Let us assume that $\mathcal{D}_{\text{dev}}$ is the development set which is utilized to assess the performance. We establish a scoring function, which is often known as an evaluation function:

$$g : Y \times Y \rightarrow \mathbb{R}.$$

Here, the value $g(\hat{y}, y)$ could represent Accuracy, F1-score, BLEU, or other metrics which are tailored to the specific task. Basically, the problem of discrete prompt

optimization is formally stated as finding the optimal prompt p^* such that:

$$p^* = \arg \max_{p \in \mathcal{P}} \sum_{(x,y) \in \mathcal{D}_{\text{dev}}} g(f(p \oplus x), y).$$

It is difficult to handle this problem if we use conventional gradient-based optimization techniques. The reason is because the space $\mathcal{P}$ is discontinuous and very large. Also, LLMs are usually accessed via "black-box" APIs, so we do not have access to internal weights of the model.

2.2 EvoPrompt and LLM-based Evolutionary Algorithms

2.2.1 EvoPrompt’s Fundamentals

EvoPrompt [3] is a framework proposed to optimize discrete prompts, which is inspired by Evolutionary Algorithms (EAs). In this method, every "individual" in the population is considered as a text prompt. Different from conventional EAs that work on bit strings or numerical vectors, EvoPrompt utilizes LLMs to perform evolutionary operators directly inside the natural language space. In particular, the LLM functions as a generative operator to perform the following actions:

- **Mutation:** Create a new variety by paraphrasing or changing an existing prompt to a different one.
- **Crossover:** Create a new offspring prompt by combining the information of two or more parent prompts together.

This method allows us to search for the best prompt efficiently by utilizing the reasoning and language generating skills of the LLM.

2.2.2 Overall Process

The workflow of EvoPrompt consists of the following steps:

1. **Initialization:** We create an initial population of prompts $\{p_1, p_2, \dots, p_m\}$ by using manual heuristics or instructions designed by human.
2. **Evaluation:** We use the development set $\mathcal{D}_{\text{dev}}$ to run each prompt through the LLM. Then, we use the function $g(\cdot)$ to calculate the score for them.
3. **Selection:** We choose the prompts that have high achievement to serve as the parents for future generation.
4. **Evolution:** We apply the Crossover and Mutation operations by using the LLM. To do this, standard methods like Differential Evolution (DE) and Genetic Algorithms (GA) can be used.
5. **Update:** In order to create the population for the next iteration, we swap out the prompts with low performance with the freshly created ones.

This cycle is repeated until the performance converges or the number of iterations reaches maximum.

2.2.3 EvoPrompt’s Limitations

Although EvoPrompt has demonstrated potential in discrete prompt optimization, however, it still has a number of important drawbacks as below:

Risk of Local Optima: The algorithm is easy to be trapped in poor solutions (local optima). This is because the diversity of population is usually lack.

Absence of Context Optimization: It ignores the possibility of in-context examples (few-shot demonstrations) and only focus on optimizing the instruction text.

High Computational Cost: Because the method necessitates to call API to the LLM many times for both prompt generation and evaluation, so it is very costly.

Algorithm 1 Discrete Prompt Optimization: EvoPrompt

Require: Initial prompts $P_0 = \{p_1, p_2, \dots, p_N\}$, size of population N , a dev set D , $f_D(\cdot)$ denotes the score of a prompt on the desired LLM evaluated on D , a pre-defined number of iterations T , carefully designed evolutionary operators to generate a new prompt $\text{Evo}(\cdot)$

- 1: Initial evaluation scores: $S_0 \leftarrow \{s_i = f_D(p_i) \mid i \in [1, N]\}$
- 2: **for** $t = 1$ **to** T **do** $\triangleright T$: number of iterations
- 3: **Selection:** select a certain number of prompts from current population as parent prompts $p_{r1}, \dots, p_{rk} \sim P_{t-1}$
- 4: **Evolution:** generate a new prompt based on the selected parent prompts by leveraging LLM to perform evolutionary operators $p'_i \leftarrow \text{Evo}(p_{r1}, \dots, p_{rk})$
- 5: **Evaluation:** $s'_i \leftarrow f(p'_i, D)$
- 6: **Update:** $P_t \leftarrow \{P_{t-1}, p'_i\}$ and $S_t \leftarrow \{S_{t-1}, s'_i\}$ based on the evaluation scores
- 7: **end for**
- 8: **Return:** the best prompt, $p^* \leftarrow \arg \max_{p \in P_T} f(p, D)$

These difficulties necessitate the development of better techniques, especially the methods that can co-optimize the few-shot instances and the prompt instructions to improve overall efficacy and stability.

3 The Proposed Method

3.1 Concepts Used

- **Few-shot Learning:** This is a common technique used to provide examples for LLMs. Providing examples helps the model understand and perform tasks more accurately, playing a crucial role in shaping the desired response.
- **Self-Explained Mechanism:** This is an important concept used in the proposal method. This mechanism helps the LLM understand the context better by requiring the LLM to generate its own explanation or reasoning for its response. This method ensures that the generated response is not just randomly generated but also based on the LLM’s understanding of the task and data.

3.2 Overview of the Proposed Method

In the original EvoPrompt method, we saw the potential for automatically developing prompts using evolutionary algorithms. However, a crucial question remains: the authors have yet to prove the superiority of using an evolutionary approach for the automatic prompt optimization problem compared to other approaches, such as local search or simply randomly changing the wording. Especially in the authors’ experiments, the initial population of prompts was often quite small, and the transformation and evolution process revolved around randomly modifying keywords. Therefore, the search space remained relatively small, failing to demonstrate the superiority of the evolutionary algorithm in finding the global optimal solution.

This motivated the research team to conduct experiments to enrich the context of the prompts, thereby expanding the search space, making it more suitable for evolutionary algorithms, and ultimately improving the results. The context enrichment method used here is inspired by the few-shot prompting technique. Specifically, at each evolutionary step, we select a sample (a pair of sentences and their corresponding results) from the dev set and send an additional request to the LLM to generate an inference sentence that matches that sample. This inference sentence is then enhanced in the child prompt to add more context.

3.3 Instantiation with Genetic Algorithm

In this study, the team improved the evolutionary mechanism of the original EvoPrompt by adding Explanation Injection. The specific process is modeled in Algorithm 2:

Algorithm 2 Discrete Prompt Optimization: EvoPrompt (GA)

Require: Initial prompts $P_0 = \{p_1, p_2, \dots, p_N\}$, population size N , dev set D

Ensure: Best prompt p^*

```

1: Initial fitness evaluation:  $S_0 \leftarrow \{s_i = f(p_i, D) \mid i \in [1, N]\}$ 
2: for  $t = 1$  to  $T$  do  $\triangleright T$ : number of iterations
3:   for  $i = 1$  to  $N$  do
4:     Selection: choose parents  $p_{r1}, p_{r2} \sim P_{t-1}$  using roulette wheel and  $qa \sim D$ 
5:     Evolution:  $p'_i \leftarrow \text{GA}(p_{r1}, p_{r2}, qa)$ 
6:     Evaluation:  $s'_i \leftarrow f(p'_i, D)$ 
7:   end for
8:    $S_t \leftarrow \{s'_i \mid i \in [1, N]\}$ 
9:    $P_t \leftarrow \{p'_i \mid i \in [1, N]\}$ 
10:  Update score:  $S_t \leftarrow \text{Top-N}(S_{t-1} \cup S_t)$ 
11:  Update population:  $P_t \leftarrow \text{Top-N}(P_{t-1} \cup P_t)$  using  $S_{t-1}, S_t$ 
12: end for
13: Return:  $p^* \leftarrow \arg \max_{p \in P_T} f(p, D)$ 

```

Selection: The research team used roulette wheel selection, one of the three selection methods of the original EvoPrompt, to select parent prompts. These prompts are sorted based on fitness scores on the dev set, so the probability of selecting a prompt is proportional to its score. The difference from the original method lies in the data preparation step for the evolution. In addition to selecting parent prompts, we randomly select a question-answer pair (q,a) from the dev set. This question pair will serve the self-explained mechanism.

Evolution: In this improved method, a new prompt is generated by combining the parent prompts and adding specific semantic explanations from the question pair. This process consists of three main steps, specifically as shown in the image 1:

1. **Crossover:** The input is the parent prompts. The LLM acts as an evolutionary algorithm to merge these two prompts, creating a new prompt. The new prompt generated by the LLM is the output of this step.
2. **Explanation Injection:** This step is the key difference between the proposed method and the original method. In this step, the LLM is required to generate an explanation of why question q leads to answer a. This explanation outlines the steps the LLM needs to take for the specific problem, providing context for the task being processed. The generated explanation is also the output of this step.
3. **Mutation:** The input to the mutation step is a combination of the intermediate prompt generated in step 1 and the explanation generated in step 2. The LLM performs the mutation to rewrite the prompt to produce the final result of the evolution step.

Update: After obtaining the prompt in the evolution step, we evaluate the prompt on the dev set. The new population is updated by merging the new and specific populations, retaining the Top-N prompts with the highest scores.

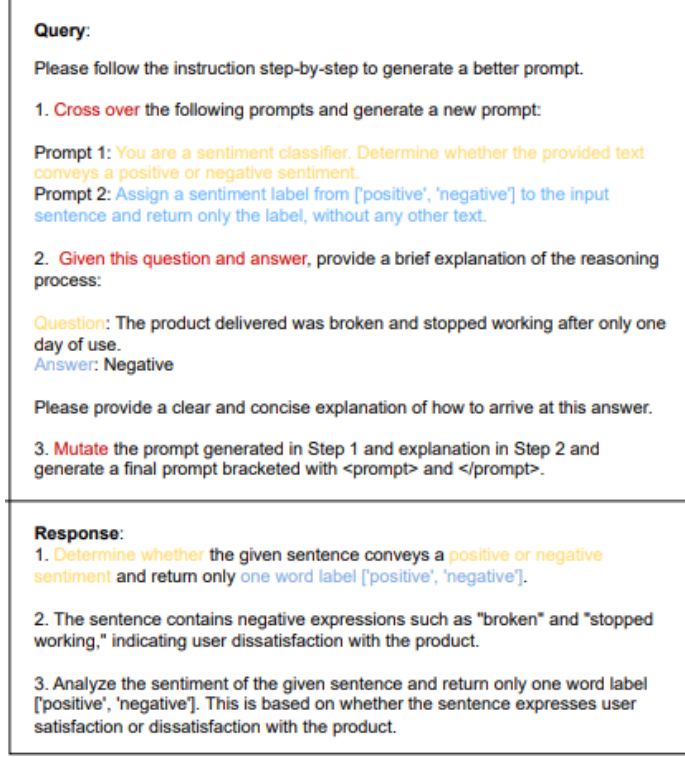


Figure 1: GA process implemented by LLMs .

3.4 Instantiation with Differential Evolution

Unlike the GA algorithm, the DE algorithm begins by iterating through each prompt in the current prompt population. This prompt p_i is called the basic prompt, and each basic prompt is executed as follows Algorithm 3:

Algorithm 3 Discrete Prompt Optimization: EvoPrompt (DE)

Require: Initial prompts $P_0 = \{p_1, p_2, \dots, p_N\}$, population size N , dev set D

Ensure: Best prompt p^*

```

1: for  $t = 1$  to  $T$  do                                     ▷  $T$ : number of iterations
2:   for each  $p_i \in P_{t-1}$  do
3:     Sample donors:  $p_{r1}, p_{r2} \sim P_{t-1}$ , where  $r_1 \neq r_2 \neq i$ 
4:     Choose a question-answer pair:  $qa \sim D$ 
5:     Evolution:  $p'_i \leftarrow \text{DE}(p_i, p_{r1}, p_{r2}, p_{\text{best}}, qa)$ 
6:     Selection:  $p_i^* \leftarrow \arg \max_{p \in \{p_i, p'_i\}} f(p, D)$ 
7:   end for
8:   Update population:  $P_t \leftarrow \{p_i^* \mid i \in [1, N]\}$ 
9: end for
10: Return:  $p^* \leftarrow \arg \max_{p \in P_T} f(p, D)$ 

```

Sampling: First, we randomly sample two prompts from the current population, referred to as donors.

Choose a question-answer pair: In this step, a question-answer pair is extracted from the dev set.

Evolution: The evolution step in DE is more complex than that in GA and consists of five stages. These stages are performed by the LLM, as illustrated in the figure 2:

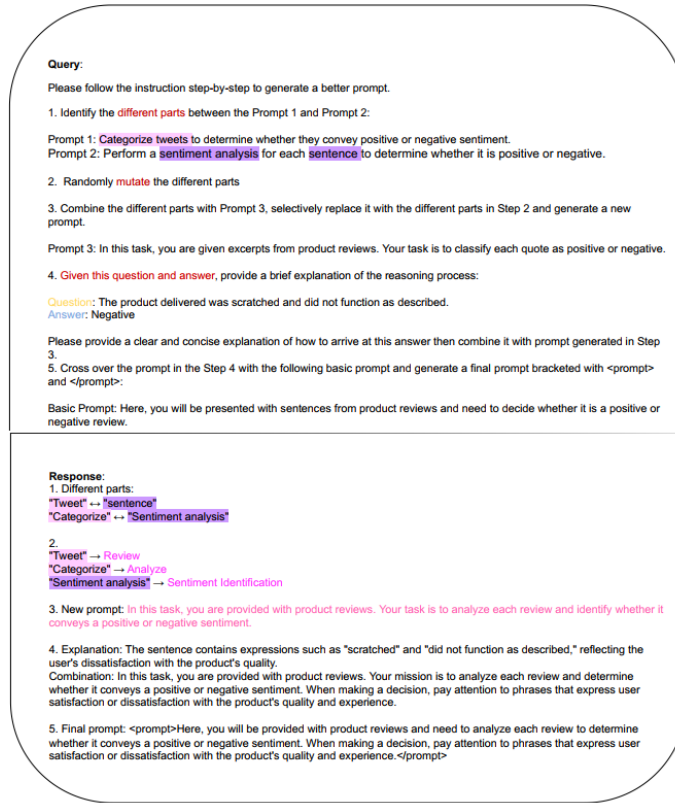


Figure 2: DE process implemented by LLMs .

1. **Identification of Differences:** The LLM is requested to compare the two randomly selected donor prompts and extract their differences. This operation replaces the traditional error-vector computation used in classical DE.
2. **Random Mutation:** The differences identified in the previous step are randomly transformed by the LLM. This operation increases the diversity of the generated prompt, which explains why DE was considered to produce a more diverse prompt population than GA in the original EvoPrompt.
3. **Combination with Best Prompt:** The mutated content obtained in the previous step is used to replace the corresponding parts of the current best prompt. Effectively, this step modifies the best-performing prompt.
4. **Explanation Injection:** This step represents a key innovation in EvoPrompt-F and is applied in a manner similar to GA. The LLM generates an explanation of the reasoning process for the selected question-answer pair, which is then appended to the prompt produced in the previous step.
5. **Crossover with Basic Prompt:** In the final stage, the prompt generated in the previous step is crossed over with the original basic prompt. This operation preserves diversity while introducing new keywords and reasoning patterns relative to the best prompt.

Selection: After the evolution step, the better-performing prompt between the original basic prompt and the newly generated prompt is selected and retained in the current population.

Update population: In the final step, the selected prompt is compared with the existing prompt population based on evaluation scores, and the Top- N highest-scoring prompts are preserved for the next generation.

Table 1: The detailed statistics of Logical Deduction Seven Objects dataset (BBH).

Attribute	Specification
Dataset name	<code>logical_deduction_seven_objects</code> (BBH)
Task type	7-way multiple-choice logical deduction
Data Format	JSON format (array <code>examples</code> with <code>input</code> , <code>target</code>)
Number of Examples	250 samples
Answer choices	7 options (A–G)
Label distribution	A:37, B:38, C:38, D:38, E:34, F:33, G:32
Language	English
Data Fields	Scenario + options (input), label (target)
Input Modality	Text only

4 Experiments

In this section, comprehensive experiments are conducted to investigate on the effectiveness of our proposed method. We aim to answer two critical question: (1) whether the proposed EvoPrompt-F approach can achieve better performance on prompt optimization task compare with the previous state-of-the-art method, and (2) how is the trade-off between the performance improvement and the additional computational cost (model calls) when comparing to original EvoPrompt framework.

4.1 Experimental Setup

4.1.1 Datasets Description

In order to evaluate the performance, we utilize dataset covering two different domains: language understanding task and logical reasoning task. To be specific, the datasets are selected as follows:

- **Language Understanding tasks:** We employ SST-2 and SST-5 datasets.
- **BBH (Big-Bench Hard) Reasoning tasks:** We select Dyck Languages, Multi-Step Arithmetic, and Logical Deduction Seven Objects.

It is worth noting that the main evaluation discussed in this paper focus on the BBH subset `logical_deduction_seven_objects`. The detailed statistical information and property of this specific dataset are listed in the Table 1. As can be seen, it contains 250 examples with 7 answer choices from A to G.

4.1.2 Models Selection

Regarding to the language understanding experiment, we follow the setting in previous works and utilize Alpaca-7B as the base model. On the other hand, for the BBH reasoning tasks, Gemma-3-12B-IT model is employed. We choose this model because it serve as a practical proxy for chat-based model and it exhibit similar behavior on the deduction capabilities.

4.1.3 Baselines and Hyperparameters Implementation

To demonstrate the superiority, we compare our proposed EvoPrompt-F with the original EvoPrompt implementation as the strong baseline. We directly use the source code provided by the authors. All experiment utilize the same set of hyperparameter for both baseline and our method to ensure the fair comparison.

The detailed parameter settings are presented in Table 2 and Table 3.

Table 2: Detailed parameter settings for BBH experiment.

Hyperparameter	Value
Population size	10
Optimization step (T)	1
Development set size (D)	50
Batch size	20
Random seed	1

Table 3: Detailed parameter settings for language understanding experiment.

Hyperparameter	Value
Population size	10
Optimization step (T)	1
Development set size (D)	200
Batch size	16
Random seed	15

In our early trial experiment, we test the optimization step T in the range of $\{1, 5, 10\}$. We observe that the method show fast convergence. Therefore, all the reported result in this paper use 1 optimization step for both the baseline and EvoPrompt-F.

4.1.4 Configuration of EvoPrompt-F

For the proposed EvoPrompt-F, we use exactly same hyperparameter configuration as the baseline model. The accuracy score is used as the main evaluation metric, which is consistent with original EvoPrompt paper.

4.2 Results and Analysis

4.2.1 RQ1: Performance Comparison Analysis

In this subsection, we discuss about the performance comparison. Figure 3 illustrates the result of baseline, and Figure 4 displays the result of EvoPrompt-F on the BBH Logical Deduction Seven Objects task.

From the results, we can observe that EvoPrompt-F achieve 73.5% accuracy when using genetic algorithm (GA). This result indicate an improvement of about 18% compared to the EvoPrompt baseline. Furthermore, with the filtering step applied, EvoPrompt-F is able to reach 77.5% accuracy, which is significantly 22% higher than baseline performance.

Based on the analysis, two important point are observed. Firstly, EvoPrompt-F is capable of generating more new prompt in the top rank list. In contrast, the baseline method often keep the initial prompt in the top rank. We can see this phenomenon in Dyck Languages dataset, where the best prompt come from the initial pool. The reason is that initial prompt is usually short and contain little task-specific signal.

Secondly, when we increase the number of sample in explanation module to 3 and combine it with the prior prompt, the performance is dropped to 71.3% under GA setting. This degradation is caused by the stochastic nature of the model response and the limited diversity of sample in the dataset.

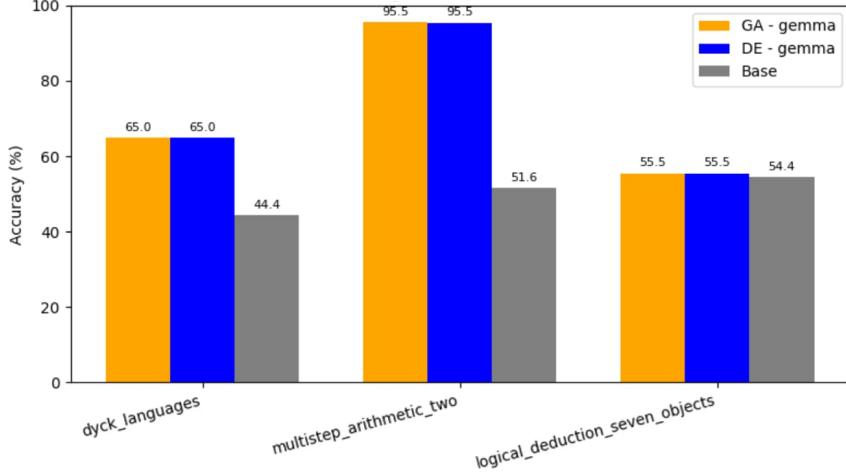


Figure 3: The performance of original EvoPrompt baseline on BBH Logical Deduction Seven Objects.

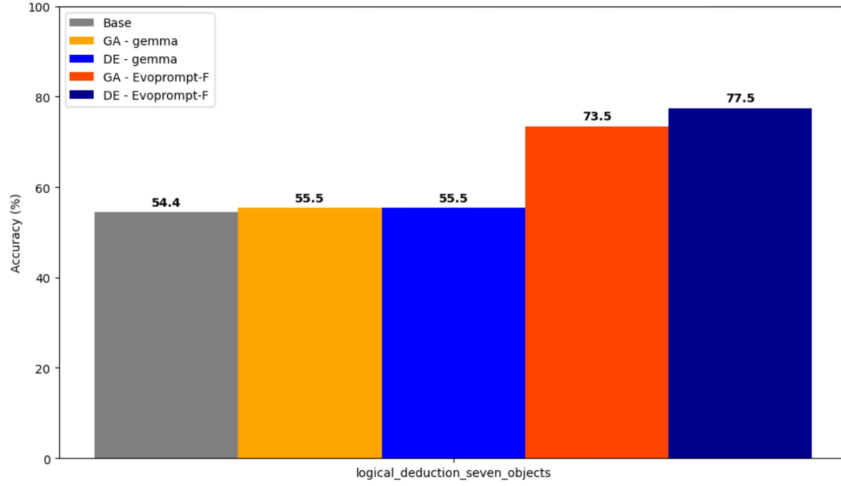


Figure 4: The performance of proposed EvoPrompt-F on BBH Logical Deduction Seven Objects.

4.2.2 RQ2: Computational Cost Analysis

We also analyze on the cost of the methods. The cost of baseline in term of LLM invocation count is expressed as the following equation:

$$\text{Cost}_{\text{EvoPrompt}} \approx N \times (D + E) \times T, \quad (1)$$

where N denotes the number of initial prompt, D represent the dev-set size per prompt, E is the number of LLM call per evolutionary step, and T is the total number of optimization step.

The proposed EvoPrompt-F add two additional LLM call per step. Specifically, one call is for generating explanation and one call is for merging explanation with the evolved prompt. This modification increase the step call count by a constant factor, however, it does not change the scaling complexity on N, D, T . Therefore, although EvoPrompt-F increase the absolute number of model call, the complexity remain unchanged.

In summary, the experiment result shows that EvoPrompt-F provides clear accuracy improvement on BBH subset with only a small constant increase in LLM usage cost.

5 Conclusion

In this paper, we have proposed a novel framework named EvoPrompt-F, which serve as an explanation-aware extension for the original EvoPrompt. We conduct the experiments to validate the effectiveness of our method on both language understanding task and reasoning benchmark. Based on the experimental result on BBH Logical Deduction Seven Objects, it is proved that EvoPrompt-F shows the significant accuracy gain compare with the baseline method. Meanwhile, it only increase the LLM call by a small constant factor. For the future work, we will plan to study on scaling the proposed approach to larger benchmark and more complex dataset.

References

- [1] Li, W., Wang, X., Li, W., Jin, B.: A survey of automatic prompt engineering: An optimization perspective. *Journal Name*
- [2] Ramnath, K., Zhou, K., Guan, S., Mishra, S.S., Qi, X., Shen, Z., Wang, S., Woo, S., Jeoung, S., Wang, Y., Wang, H., Ding, H., Lu, Y., Xu, Z., Zhou, Y., Srinivasan, B., Yan, Q., Chen, Y., Ding, H., Xu, P., Cheong, L.L.: A systematic survey of automatic prompt optimization techniques. *Journal Name*
- [3] Guo, Q., Wang, R., Guo, J., Li, B., Song, K., Tan, X., Liu, G., Bian, J., Yang, Y.: Evoprompt: Connecting llms with evolutionary algorithms yields powerful prompt optimizers. *Journal Name*